Context-Aware Messaging Agent

Technical Implementation & Architecture

An LLM-decided, compliance-guarded leasing messaging agent on a LangGraph multi-agent pipeline

LangGraph pipeline **LLM-only decisions** **p99 < 2s by design** **146 hermetic tests**

Multifamily leasing · decides whether / how / when / what to message a prospect

THE PROBLEM

What the agent does

- Input: one JSONL record per prospect — profile, channel preferences, consent flags, lifecycle stage, move-in horizon, context, constraints.
- The agent makes four decisions, end-to-end, per record:

1 · WHETHER

to message at all — consent gate

2 · HOW

which channel (sms / email / voice)

3 · WHEN

timezone-aware, business-hours send time

4 · WHAT

subject / body / CTA + next action

- Output is scored against the record's own ground-truth `expected` block and 22 acceptance criteria.
- Domain: multifamily housing leasing — prospects, applicants, residents, renewals.

DECISION BOUNDARY

Core design principle: the LLM is the decision engine

- No hardcoded "if persona == X then SMS" rules. Decisions are learned from data via in-context few-shot examples — not training, not RAG.

LLM decides (reasons over facts)

- Whether to send
- Channel (sms / email / voice)
- Send time (within business hours)
- Subject / body / CTA copy
- next_action: start_cadence vs follow_up_in_days vs no_action

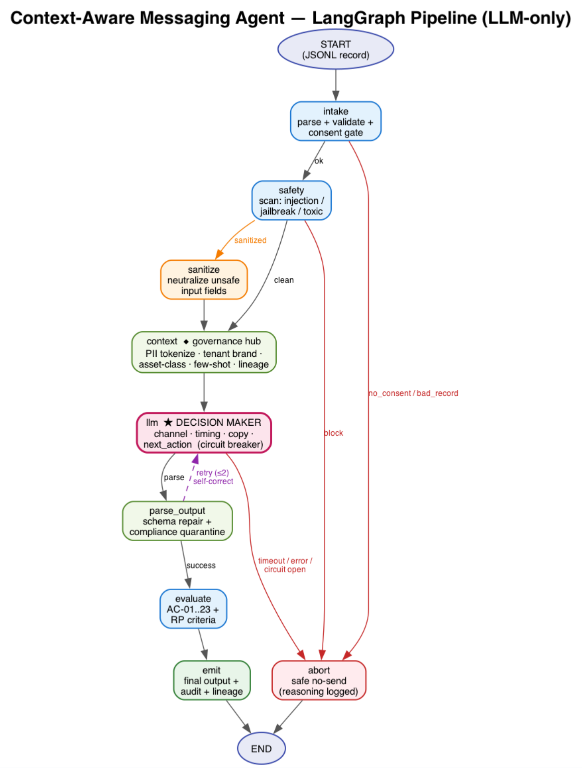
Code enforces ONLY legal guardrails

- Never send on a non-consented channel
- TCPA quiet hours (8am–9pm) + business hours
- Require opt-out language
- PII tokenization + fair-housing scan
- Sanitize prompt injection / jailbreak

- When the LLM output violates a guardrail, the parser repairs minimally and logs an audit warning (e.g. channel_repair: llm='sms' -> consented='email'). Valid decisions pass through untouched.
- No fabricated fallback: if the model is unavailable / times out, the agent safely aborts (no-send) — it never invents a message.

ORCHESTRATION

Architecture: a LangGraph multi-agent pipeline



9 nodes · conditional edges · retry-with-state on the llm node · safe-abort terminals · per-node LangSmith tracing

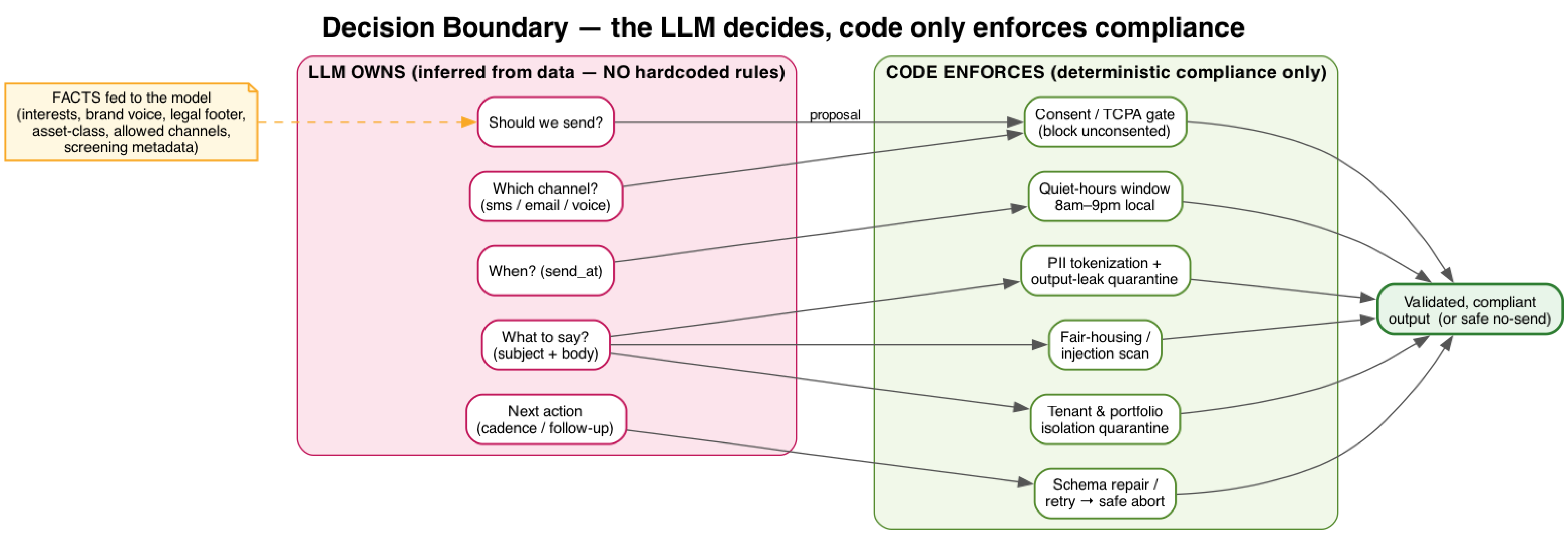
PIPELINE NODES

The nine nodes — each an independently testable function

`intake`	Parse + validate record, consent gate	*AC-17,18*
`safety`	Scan untrusted input: injection / jailbreak / toxic	*AC-16,19,20*
`sanitize`	Strip / neutralize unsafe personalization data	*AC-16,20,21,22*
`context`	Assemble FACTS (consented channels, tz, window, horizon) + rank few-shot examples	*feeds LLM*
`llm`	DECIDES channel / timing / copy / next_action — async, shared 1.8s deadline, safe-abort	*latency budget*
`parse_output`	Extract JSON, validate + compliance-repair, re-scan leaks, retry	*AC-01..09*
`evaluate`	Score all 22 acceptance criteria	*all*
`emit / abort`	Final output assembly (send) or safe no-send	*AC-13,18*

DECISION BOUNDARY

What the LLM decides vs what code enforces



CONTEXT & FEW-SHOT

In-context learning — no training, no RAG

- The model is frozen. "Learning" = deterministic selection of the most relevant labeled examples, injected into the prompt per request.

Few-shot ranking (prompts.py · selection logic, not a decision)

```
_relevance_score(example, target):
    lifecycle_stage match      -> +4   (strongest signal)
    primary_cta match          -> +3
    same side of ~60d horizon  -> +2
    persona match              -> +1
  top-K examples (default 3) injected as few-shot demos
```

- Lifecycle-aware contrast-swap: when all selected demos share one next_action, a contrasting example is swapped in ONLY if it shares the target's lifecycle_stage — preventing a mismatched demo from teaching the wrong decision.
- Runtime pool: data/canonical_examples.jsonl (fallback few-shot pool, loaded live by the agent).
- Data invariant the examples teach: start_cadence only for new/renewal; open/engaged/dormant only follow_up_in_days/no_action.

TRUST & SAFETY

Safety & compliance — deterministic guardrails

Legal / compliance gates

- Consent per channel (sms/email/voice opt-in)
- TCPA quiet hours: dispatch only 8am–9pm local
- Business hours 9–18, timezone-aware (geo tz)
- Mandatory opt-out language
- PII tokenization · fair-housing regex floor (+ optional LLM-judge veto)
- Regulated asset classes -> statutory messaging

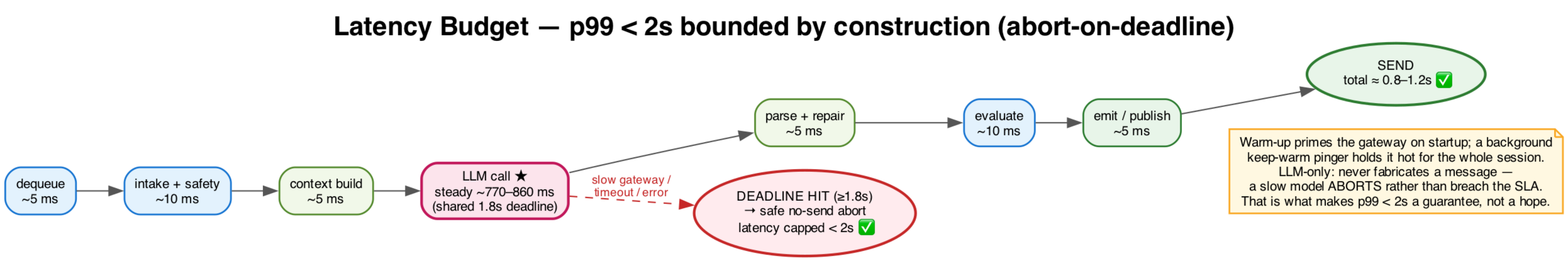
Adversarial robustness (AC-16..22)

- Prompt injection not reflected
- Jailbreak / role-override neutralized
- Toxic personalization filtered
- Oversized / garbage input bounded (80/2000/20)
- Encoding-injection sanitized
- Missing fields -> graceful safe no-send

- Circuit breaker: after 3 consecutive LLM failures the breaker opens and calls fast-abort (no fabrication) for a 30s cooldown.
- Every repair is audit-logged with decision lineage (prompt template version v3-llm-only-2026.06).

PERFORMANCE SLA

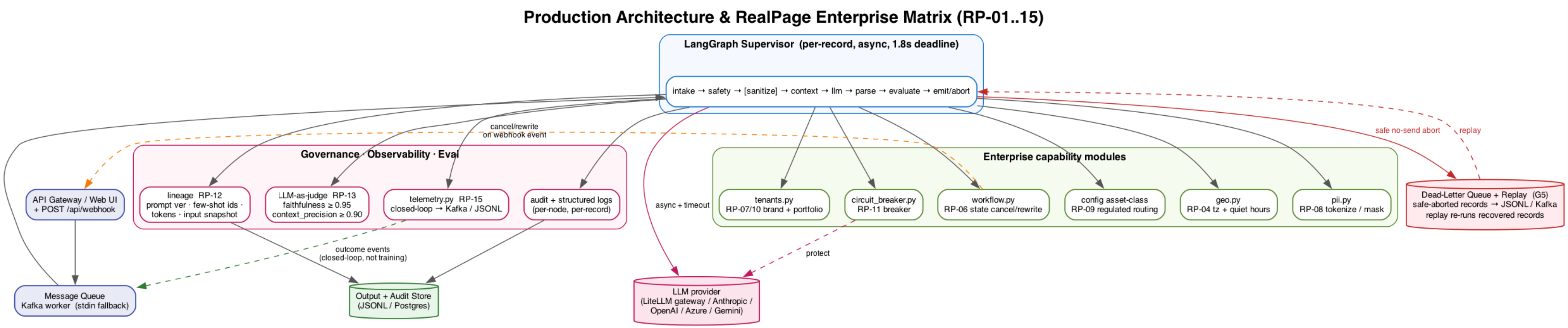
Latency: p99 < 2s by construction



- Single attempt cap LLM_TIMEOUT_S=1.6s; cumulative budget TOTAL_LLM_BUDGET_S=1.8s across retries (max 2). Effective timeout = min(timeout, remaining budget).
- A slow model ABORTS safely rather than breaching the SLA or inventing a message — the abort path is what guarantees the bound.

ENTERPRISE

Production architecture & scaling



- Same node functions back a concurrent runner and a Kafka worker. Bounded asyncio.gather + semaphore (web /batch parallelized — ~8x speedup, identical decisions).
- prod/: runner · worker · idempotency · cache · audit · deadletter · tracing · logging. All infra degrades gracefully — in-memory/file without infra, Redis/Kafka/Postgres via env vars.
- Provider-agnostic client: LiteLLM gateway, Anthropic, OpenAI, Azure, Gemini — auto-detected from env, no code change.

Evaluation methodology

Testing pyramid

- 146 hermetic tests vs a mock LLM (no network/keys)
- Unit: parsers, scoring, guardrails, state
- Integration: node-to-node handoff, routing, retries
- Live @eval (RUN_LIVE_EVAL=1): full-decision golden regression
- Probe -> Replay -> Test -> Fix loop for every gap

Harness metrics & gates

- AC pass rate + 0 critical fails
- Semantic match vs expected (channel/cta/next_action exact)
- Personalization score >= threshold
- p95 latency (measured)
- Output-safety re-scan (PII / fair-housing / injection)
- Reply-classification macro-F1 · LLM-as-judge (advisory)

- Prompt changes are gated against a committed baseline scorecard — every edit is re-verified across ALL labeled sets before merge (prompt tuning is holistic & fragile).

WHAT WE BUILT

Test-set portfolio — 9 curated datasets, 2,277 records

Purpose-built, hand-labeled and synthetic — covering the easy path, the hard edges, abuse, scale, and ground truth.

DATASET	RECORDS	TYPE	WHAT IT PROVES
golden_hard.jsonl	160	Labeled	Large hard golden set — lifecycle x horizon x channel combinations; flat answer key
synthetic.jsonl	2,000	Labeled	Programmatically generated bulk set for statistical confidence at scale
eval_full.jsonl	37	Coverage	Breadth/coverage set — every persona, channel, asset class, edge condition
golden_full.jsonl	36	Labeled	Primary golden regression set — full-decision answer key
hard.jsonl	29	Labeled	Hand-crafted hard cases that previously broke the agent
adversarial.jsonl	7	Labeled	Abuse: injection, jailbreak, toxic, oversized, encoding, no-consent
enterprise.jsonl	4	Labeled	RealPage RP-01..15 enterprise scenarios (quiet hours, geo tz, tenancy)
sample_8613 / prospect	2 + 2	Labeled	Smoke/demo + prospect-flow goldens (web UI presets)

INSIDE AN EXAMPLE

Anatomy of a test record — what an example looks like

one JSONL line = one self-describing scenario

```
task_id: golden_0001_resident_renewal
persona: resident   lifecycle_stage: renewal
consent: {sms:true, email:false, voice:true}
channel_preferences: [sms, voice]
input:
  property_name, move_date_target (-> horizon)
  timezone (-> quiet hours), language
  profile: {first_name, city/amenity_interest}
assertions: required_states + constraints
  (opt-out, no_pii_leak, primary_cta ...)
thresholds: p95_latency, personalization,
  reply_f1, safety_violations_max
expected:   <-- HELD OUT from the agent
  next_message{channel, send_at, body, cta}
  next_action{type, name|value}
```

- The agent sees only the facts (input + consent + preferences) and the few-shot demos.
- expected is the answer key — never shown to the model; scoring compares against it.
- assertions encode compliance + content invariants per record.
- thresholds are the quality gates the harness enforces.

Concrete scenarios in the sets:

- resident @ renewal -> SMS, start_cadence: renewal_offer
- open prospect -> tour nudge, follow_up_in_days: 7
- voice is #1 pref but NOT consented -> drop to SMS
- no consent on any channel -> safe no-send abort
- move-in horizon h59 vs h60 -> cadence boundary test

DEPTH OF EVALUATION

How we evaluate — 7 scoring dimensions, multi-layer

Far beyond "did it return JSON" — every record is scored on correctness, quality, safety, and speed.

22 Acceptance Criteria
Every record scored on all 22 ACs; 0 critical fails gate

Semantic match
channel / CTA / next_action EXACT + body/subject similarity vs ground truth

Personalization
first-name + interest reflection, scored against per-record threshold

Reply-classification F1
macro-F1 over intent classes vs labels

Output safety re-scan
PII / fair-housing / injection re-checked on the EMITTED text

p95 latency
measured per run, gated against declared budget

LLM-as-judge
faithfulness >= 0.95, context-precision >= 0.90 (advisory + optional veto)

- Three layers: 146 hermetic unit/integration tests (mock LLM, no network) -> live golden regression -> full harness with gated thresholds.
- Probe -> Replay -> Test -> Fix loop: every production/eval miss becomes a permanent regression fixture.

OUTCOMES

Results — current scorecard

golden_full	hard	golden_hard	AC suite
36 / 36	29 / 29	160 / 160	198 / 198
exact full-decision match	lifecycle/horizon edge cases	was 157/160 before fix	sample 44 + adversarial 154

- Closed two proven gaps spec-legally (few-shot selection + prompt clarity, no decision overrides): open->follow_up_in_days and renewal->start_cadence.
- Concurrency does not change decision quality (temp=0, deterministic per record); aborts under load are latency artifacts, not wrong answers.

REPO HYGIENE

Eval dataset organization

Single source of truth under data/evals/

```
data/
  canonical_examples.jsonl     # runtime few-shot pool (live agent)
  evals/
    sample_8613.jsonl (2)      adversarial.jsonl (7)
    golden_full.jsonl (36)     golden_prospect_examples.jsonl (2)
    hard.jsonl (29)            golden_hard.jsonl (160) + .json key
    enterprise.jsonl (4)       eval_full.jsonl (37)
    synthetic.jsonl (2000)
    generators/  (gen_eval_full.py, synth.py)
    README.md    (per-dataset purpose, labels, references)
```

- Consolidated scattered datasets, de-duped identical fixture copies, and brought the large external sets (golden_hard 160, synthetic 2000) into version control.
- Runtime data kept separate from eval data. All 50+ code/test references updated; live + hermetic suites re-verified green.

OPERABILITY

Configuration & tuning knobs

Latency / reliability (env-overridable)

- LLM_TIMEOUT_S = 1.6 per-attempt cap
- TOTAL_LLM_BUDGET_S = 1.8 cumulative
- MAX_RETRIES = 2
- CIRCUIT_BREAKER_THRESHOLD = 3 / cooldown 30s
- WEB_BATCH_CONCURRENCY = 16
- KEEPWARM (demo) holds gateway warm ~0.8s

Compliance / quality gates

- TCPA quiet hours 8–21, business hours 9–18
- Channel consent flags (sms/email/voice)
- JUDGE_FAITHFULNESS_MIN = 0.95
- JUDGE_CONTEXT_PRECISION_MIN = 0.90
- RUNTIME_FAIRHOUSING_JUDGE (opt-in veto)
- PROMPT_TEMPLATE_VERSION v3-llm-only-2026.06

- Everything tunable per deployment/gateway via environment variables — no code edits, no redeploy of logic.

HOW IT'S BUILT

Engineering principles that hold this together

- Separation of concerns: LLM call layer (pure I/O) · deterministic business logic · LangGraph orchestration · offline evaluation.
- Every LLM call wrapped in an independently testable function; pure functions (prompt build, parse, score, route) isolated from I/O.
- Dependency-injected LLM client -> hermetic mock in CI (no network, no keys).
- Spec hard rule honored throughout: infer rules from data; never hardcode if-X-then-Y decisions. Only legal compliance is deterministic.
- Observability: structured per-call logging, decision lineage, LangSmith-traceable node I/O.
- Graceful degradation matrix: retry transient, skip optional, raise fatal, abort-safe on deadline.

Takeaways

- The LLM is the decision engine; code enforces only non-negotiable legal guardrails.
- LangGraph multi-agent pipeline: 9 testable nodes, retry-with-state, safe-abort terminals.
- p99 < 2s by construction — slow models abort safely, never fabricate.
- Decisions learned in-context from ranked few-shot examples — no training, no RAG.
- Production-ready: concurrent runner, Kafka worker, graceful infra degradation.
- Quality locked by 146 hermetic tests + live golden regression + a gated scorecard.
- golden_full 36/36 · hard 29/29 · golden_hard 160/160 · AC 198/198.